Building PES-VCS — A Version Control System from Scratch

Operating System

Name : Prakash N Heroor
SRN : PES1UG24CS595
Section : J
Subject : Operating System

GITHUB LINK : https://github.com/prakashheroor89-stack/PES1UG24CS595-pes-vcs

📸 **Screenshot 1A:** Output of ./test_objects showing all tests passing.

```
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$
```

Screenshot 1B: find .pes/objects -type f showing the sharded directory structure.

```
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$
```

Screenshot 2A: Output of ./test_tree showing all tests passing.

```
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$ find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$ find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
```

Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$ ./pes init
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_objects.c
  untracked:  object.c
  untracked:  Makefile
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  test_tree
  untracked:  hello.txt
  untracked:  commit.c
  untracked:  test_objects
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  commit.h
  untracked:  pes.c
  untracked:  README.md
  untracked:  index.c
  untracked:  tree.h
  untracked:  index.h

prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$ cat .pes/index
304444 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776415750 6 file1.txt
304444 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776415750 6 file2.txt
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$
```

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$ ./pes log
commit e3a2dd6270c511441aff133eb167d32f45e1e2a162db96d253444b42dc8f0da0
Author: PES User
Date:   0

    Second commit


commit ca8483d14cec23cc0767dbd772abaea17a47c47308f52d10f23d0b10cd36eb25
Author: PES User
Date:   0

    Initial commit
```

📌 **Screenshot 4B:** find .pes -type f | sort showing object store growth after three commits.

```
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6f/ba30d9c432b5df94416b2a884d11497f779f89d3eab02e88e1b16fede48caf
.pes/objects/ca/8483d14cec23cc0767dbd772abaea17a47c47308f52d10f23d0b10cd36eb25
.pes/objects/de/db71b1456f49497b7c3ee17e0764d7cc1f7b3cad200799b545a7a24cd8128f
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e3/a2dd6270c511441aff133eb167d32f45e1e2a162db96d253444b42dc8f0da0
.pes/refs/heads/main
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$
```

📌 **Screenshot 4C:** cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$ cat .pes/refs/heads/main
cat .pes/HEAD
e3a2dd6270c511441aff133eb167d32f45e1e2a162db96d253444b42dc8f0da0
ref: refs/heads/main
prakash@PRAKASH06:~/PES1UG24CS595_Prakash_ORANGE$
```

📌

Q5.1 — Branch checkout design

Question:
A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Answer:

A branch in PES-VCS is represented as a file under .pes/refs/heads/ containing the latest commit hash. The HEAD file stores a reference to the active branch.

To implement pes checkout <branch>, the following steps are required:

1. Verify that the branch file .pes/refs/heads/<branch> exists.
2. Update .pes/HEAD to point to the new branch:

    - Write: ref: refs/heads/<branch>
3. Read the commit hash from the branch file.
4. Load the commit object and retrieve its tree.
5. Traverse the tree recursively and reconstruct the working directory:
    - For each blob, write its contents to the corresponding file path.
    - For each tree, create directories accordingly.
6. Update the index to match the checked-out tree.

This operation is complex because:

- The working directory must exactly match the snapshot.
- Existing files may need to be overwritten or deleted.
- Conflicts must be handled if local changes exist.
- Recursive tree traversal is required for nested directories.

Q5.2 — Dirty working directory detection Question:

When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Answer:

A dirty working directory occurs when files in the working directory differ from the index.

To detect this:

1. Iterate through all entries in the index.
2. For each file:
    - Read file content from disk.
    - Compute its SHA-256 hash using the same method as object_write.
3. Compare the computed hash with the stored hash in the index.
    - If hashes differ → file is modified.
4. Also check:
    - If a file exists in index but not on disk → deleted.
    - If a file exists on disk but not in index → untracked.

Checkout must be aborted if:

- Any tracked file is modified and differs from target branch content.

This ensures no local changes are overwritten unintentionally.

Q5.3 — Detached HEAD Question:

"Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Answer:

Detached HEAD occurs when .pes/HEAD contains a direct commit hash instead of a branch reference.

In this state:

- New commits are created normally.
- However, no branch pointer is updated.
- These commits become "unreachable" from any branch.

Consequences:

- The commits exist in the object store but are not referenced.
- They may be lost during garbage collection.

Recovery:

- The user can create a branch pointing to the current commit:
  pes branch <new-branch>
- Alternatively, manually write the commit hash into a branch file under .pes/refs/heads/.

Detached HEAD is useful for temporary exploration but unsafe for long-term work.

Q6.1 — Garbage Collection Question:

Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Answer:

Garbage collection identifies and removes objects not reachable from any reference.

Algorithm:

1. Start from all references:
   - .pes/refs/heads/*
   - HEAD commit
2. Traverse recursively:
   - Commit → Tree
   - Tree → Blobs and subtrees
3. Store all visited hashes in a HashSet (or bitmap).
4. Scan .pes/objects/:
   - For each object:
     - If not in reachable set → delete it.

Data structure:

- HashSet is ideal for O(1) lookup.

Complexity:

- If 100,000 commits and 50 branches:
  - Worst case: traverse all commits and trees
  - ~100k–300k objects visited

This ensures only actively referenced data is preserved.

Q6.2 — GC race condition

Question:

Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Answer:

Garbage collection is dangerous during concurrent commit operations due to race conditions.

Scenario:

1. A commit is being created:
   - Blob and tree objects are written.
   - Commit object is not yet linked to any branch.
2. GC runs simultaneously:
   - It scans reachable objects.
   - The new commit is not yet referenced → considered unreachable.
3. GC deletes the newly created objects.
4. Commit process finishes → now references deleted objects.

Result:

- Repository corruption.

Git avoids this using:

- Atomic reference updates

- Lock files during commit and GC
- Delayed pruning (grace period before deletion)
- Mark-and-sweep with safety checks

Thus, GC never deletes objects that might still become reachable.